

Calowin is designed to encourage environmentally friendly behavior and promote personal health. Unlike traditional navigation apps, Calowin offers route planning while focusing on environmental and fitness tracking. It calculates carbon savings and calories burned when visiting various green areas across Singapore, supporting sustainability and active lifestyles.

Our development approach leveraged the Agile methodology, which ensured continuous improvement through an iterative process. We utilized the SCRUM framework to foster collaboration and maintain a user-centric design, allowing us to adapt effectively based on user feedback. We use Flutter for the front end and Java 17, Spring, and Microsoft SQL server for the backend implementation.

The main functions of our application are as follows: The *Accounts* module handles essential tasks such as user registration, login, password recovery, and an OTP-based email service to ensure secure access. The *User Profile* module enables users to manage their personal details, allowing them to view and edit their profile, change their password, or delete their account if necessary. The *Trips* module leverages the Google Map API for trip management, covering initiation, cancellation, completion, and real-time metric calculations, including carbon savings and calories burned. The *Wellness Zones* module integrates NParks and live weather APIs, helping users discover and visit optimized wellness locations in Singapore. The *Friends* module enhances social interaction by allowing users to add friends, manage connection requests, and view friends' profiles. Finally, the *Leaderboard* module supports healthy competition by displaying rankings that users can filter based on activity levels, fostering motivation and engagement within the app's community.

We utilized an implementation structure very similar to microservices. We have four main groups of functionalities here. The spring node module interacts with these four services while interacting with the front end. The next slide is an example to show case our microservice architecture. The *Controller* package, like `HttpReqController.java`, serves as the interface between the client and the back-end logic. It processes incoming requests, handles user input, and communicates with the service layer to return responses.

For each service, we all use the layered architecture. At the top, the *Presentation Layer* features various boundary classes such as `SignUpUI`, `LoginUI`, `MapUI`, which act as user interfaces that facilitate interaction between users and the application. The *App Logic Layer* contains control classes like `AuthenticationManager`, `MetricsCalculator`, `TripManager`, and others that handle the core business logic and application processes. These managers coordinate interactions between the UI and the underlying data models, ensuring a smooth flow of data and responses.

Moving down to the *Object Layer*, this part of the architecture includes entity classes such as `Trip`, `CarbonSaved`, `CaloriesBurnt`, `User`, and `Achievement`. These objects represent essential data structures that interact with various control classes to execute application logic. For instance, the `Trip` entity interfaces with classes like `MetricsCalculator` and `TripManager` to provide details about trips and associated metrics like carbon savings.

Finally, the *Persistent Data Layer* manages data storage and retrieval, represented by the `AppDatabase` and `AccountDatabase`. This layer ensures that user information, trip data,

achievements, and notifications are stored and can be accessed seamlessly as needed, creating a robust data handling backbone for the app.

Design principles like loose coupling, high cohesion, separation of concerns, and the open-closed principle are central to our app. These principles contribute to a robust and flexible codebase, making it easy to extend and maintain. The *Calowin* application is built upon essential design principles that ensure its robustness, scalability, and maintainability. One key principle is **loose coupling**, exemplified by the DataTransferObject (DTO) package. This allows various parts of the system to communicate with minimal dependencies, promoting flexibility and easy modifications. **Separation of concerns** is evident in the organization of Entities, Managers, and Services within the *CalowinAccount* module, where classes like *AccountManagementService.java* and *ProfileManagementService.java* each handle distinct responsibilities. The **high cohesion** principle is demonstrated by these classes, which group related functions to enhance maintainability and clarity. Finally, the **open-closed principle** is showcased in the Database package, where new repositories such as *AchievementRepository.java* and *UserInfoRepository.java* can be added seamlessly without altering existing code. Together, these principles create a well-architected system that is both adaptable and stable, ensuring *Calowin* remains efficient and extensible over time.

The *Calowin* application leverages key object design patterns to enhance its adaptability and efficiency. The *Factory Design Pattern* allows for the flexible creation of objects, supporting easy system extension. The *Strategy Design Pattern* promotes modular code by enabling interchangeable algorithms, as seen with the *TransactionManager* managing connections to CalowinDB and CalowinSecureDB. *Dynamic Loading* optimizes performance by loading classes at runtime. We also utilize the *Facade Design Pattern* to simplify complex subsystem interactions by providing a unified interface. In this example, the Facade part acts as an intermediary between the `_handleLogin` process and underlying components like *PasswordEncoder* and *JpaRepository*. This design streamlines the login process, making it easier for the system to manage authentication while encapsulating the complexity of interacting with various services and interfaces.

To summarize, the *Calowin* application emphasizes high maintainability through its design principles of high cohesion and loose coupling. Its microservices architecture ensures scalability, while comprehensive test cases contribute to strong testability. The system's extensibility allows new features to be added without major code changes, and the use of object design patterns supports code reusability, making the application both robust and adaptable.